

# What is validated?

- (i) Functional requirements
- (ii) Non functional requirements
- (iii) Constraints & assumptions
- (iv) Interface requirements
- (v) Domain/business rules
- (vi) Models and diagrams (DFD, Use case, UML)
- (vii) Entire SRS document

# Requirement Validation Techniques -

- (i) Informal review
- (ii) Walkthrough
- (iii) Technical review
- (iv) Formal Inspection

# Output of Requirement Validation -

- (i) List of errors & inconsistencies found
- (ii) Corrected & updated SRS
- (iii) Review report
- (iv) Approval from stakeholders
- (v) Baseline SRS ready for design phase

Q. Difference between verification & validation

Verification

- 1) Are we building the product right?
- 2) Checks compliance with specification
- 3) Internal process
- 4) Ensures ~~com~~ correctness

Validation

- 1) Are we building the right product?
- 2) Checks if it meets user needs
- 3) Involves users & stakeholders
- 4) Ensures usefulness

Q. Difference between Analysis & Specification.

Analysis

- 1) Understanding and refining requirements
- 2) Detects conflicts & ambiguities
- 3) Uses models/diagrams
- 4) Focused on "analysis of needs"
- 5) Output: models, refined requirements

Specification

- 1) Documenting refined requirements
- 2) Ensures clarity and precision
- 3) Uses structured written format (SRS)
- 4) Focuses on "documentation of needs"
- 5) Output: SRS document



## # Requirement Analysis -

- 1) Understand user needs in detail
- 2) Remove ambiguities & inconsistencies
- 3) Categorize requirements (functional / non functional)
- 4) Prioritize system features
- 5) Detect conflicts among stakeholder requirements
- 6) Develop models to represent system behavior

### Activities Outputs:

- Requirement models
- Refined requirement list
- Conflicts resolved
- Prioritized requirements
- Glossary of terms
- Problem statement document

### Activities:

- Requirement modeling
- Classification: Functional & non Functional
- Conflict resolution
- Feasibility analysis
- Prioritization

## # SRS -

### Contents (IEEE Format):

- 1) Introduction
  - Purpose
  - Scope
  - Definitions
  - References
  - Overview
- 2) Overall description
  - Product perspective
  - Product features
  - User characteristics
  - Operating environments
  - Constraints
  - Assumptions

### 3) Requirement types

- Functional requirements
- Non Functional requirements
- Interface requirements
- External inputs / outputs
- Performance, security, usability constraints

### 4) Appendices

### 5) Index

### Importance:

- (i) Prevents rework
- (ii) Ensures correct system development
- (iii) Supports planning & testing
- (iv) Reduces project risks



# #Case Studies: Requirements in Online payment systems in software engineering

- Online Payment System (OPS) & allows users to make digital financial transactions using:
  - (i) Debit/Credit cards
  - (ii) Net banking
  - (iii) UPI/Wallets
  - (iv) QR based payments
  - (v) Payment gateways
- Req. study ensured the system is ~~secured~~ secure, reliable, fast and user friendly

## - Stakeholders:

- Customers
- Merchants/Sellers
- Banks & financial institutions
- Payment Gateway providers
- System administrators
- Business managers
- Security & compliance authorities

## - Functional Requirements:

- (a) User account & authentication  $\Rightarrow$  User registration & login  $\Rightarrow$  OTP based verification  $\Rightarrow$  Password reset, 2 factor authentication
- (b) Payment Processing  $\Rightarrow$  Initiate payment  $\Rightarrow$  select payment method  $\Rightarrow$  payment gateway integration  $\Rightarrow$  real time status updates  
 $\Downarrow$   
Transaction confirmation message
- (c) Wallet/UPI Support  $\Rightarrow$  Add money to wallet  $\Rightarrow$  UPI IP verification  $\Rightarrow$  Instant transfer
- (d) Transaction management  $\Rightarrow$  Generate a unique transaction ID  $\Rightarrow$  Transaction history for users  $\Rightarrow$  Refund/change back processing  $\Rightarrow$  Download receipt/invoice
- (e) Merchant Interface  $\Rightarrow$  Dashboard for transaction viewing  $\Rightarrow$  Settlement reports  $\Rightarrow$  API integration
- (f) Notifications  $\Rightarrow$  Successful payments  $\Rightarrow$  Failed payments  $\Rightarrow$  Refunds  $\Rightarrow$  Suspicious activity



## - Non Functional Requirements -

- (a) Performance Requirements  $\Rightarrow$  Payment completion within a few seconds  $\Rightarrow$  High speed gateway response  $\Rightarrow$  Supports large number of concurrent users
- (b) Security Requirements  $\Rightarrow$  PCI-DSS Compliance  $\Rightarrow$  Encryption of card data  $\Rightarrow$  Tokenization  $\Rightarrow$  Secure authentication  $\Rightarrow$  Fraud detection system
- (c) Availability and reliability  $\Rightarrow$  24/7 system uptime  $\Rightarrow$  Fail over servers  $\Rightarrow$  Backup & recovery
- (d) Usability requirements  $\Rightarrow$  Easy to use interface  $\Rightarrow$  Minimal steps for payment  $\Rightarrow$  Clean error messages
- (e) Scalability  $\Rightarrow$  Support increasing users  $\Rightarrow$  Cloud based auto scaling

## - System constraints:

- (i) Must comply with RBI guidelines
- (ii) Must use HTTPS and TLS for all transactions
- (iii) Third party gateway availability affects operations
- (iv) Bank ~~the~~ server downtime may delay transactions

## - Requirement challenges:

- (i) High concurrency requirement
- (ii) High security risks
- (iii) Network failures or bank downtime
- (iv) Handling failed or pending transactions
- (v) Managing funds refunds & reversals
- (vi) Payment gateway dependency

## - Importance of Requirement Analysis:

- (i) Prevents financial losses
- (ii) Ensures user trust & system reliability
- (iii) ~~Helps~~ Ensures legal & financial compliance
- (iv) Helps design secure transaction flow
- (v) Improves overall user experience



- Output:
- (i) Functional requirement specification
  - (ii) Non functional requirement document
  - (iii) Use case diagrams
  - (iv) Data flow diagrams
  - (v) User interface mockups
  - (vi) Risk assessment report
  - (vii) Final SRS document
- 

## # Waterfall model -

- Linear, sequential approach
- Each phase must be completed before the next begins
- No overlap among phases

- Advantages:
- Simple & easy to understand
  - Well structured documentation
  - Suitable for projects with stable requirements

- eg. Payroll Management System, where requirements are fixed & well known
- 

## # Spiral model -

- Iterative + Risk driven
- Focuses heavily on risk analysis
- Each loop represents a development phase

### - Phases:

- (i) Objectives identification
- (ii) Risk analysis
- (iii) Development and testing
- (iv) Planning the next iteration

### - Disadvantages:

- NO user feedback during development
- Difficult to make changes
- Late testing increases risk



# ASSIGNMENT-1

- 1) Explain in detail about SDLC models with figures
  - (i) Waterfall
  - (ii) Incremental
  - (iii) Spiral
  - (iv) V-model
- 2) Perform a case study on Blood Bank management system for requirement analysis/SRS
  - (i) Introduction
  - (ii) Requirements
  - (iii) Functional & non/functional diagram requirements
  - (iv) atleast 1 UML diagram

Deadline: 13 Feb 2026 (Friday)  
Just after MST-01

## #Rapid software development -

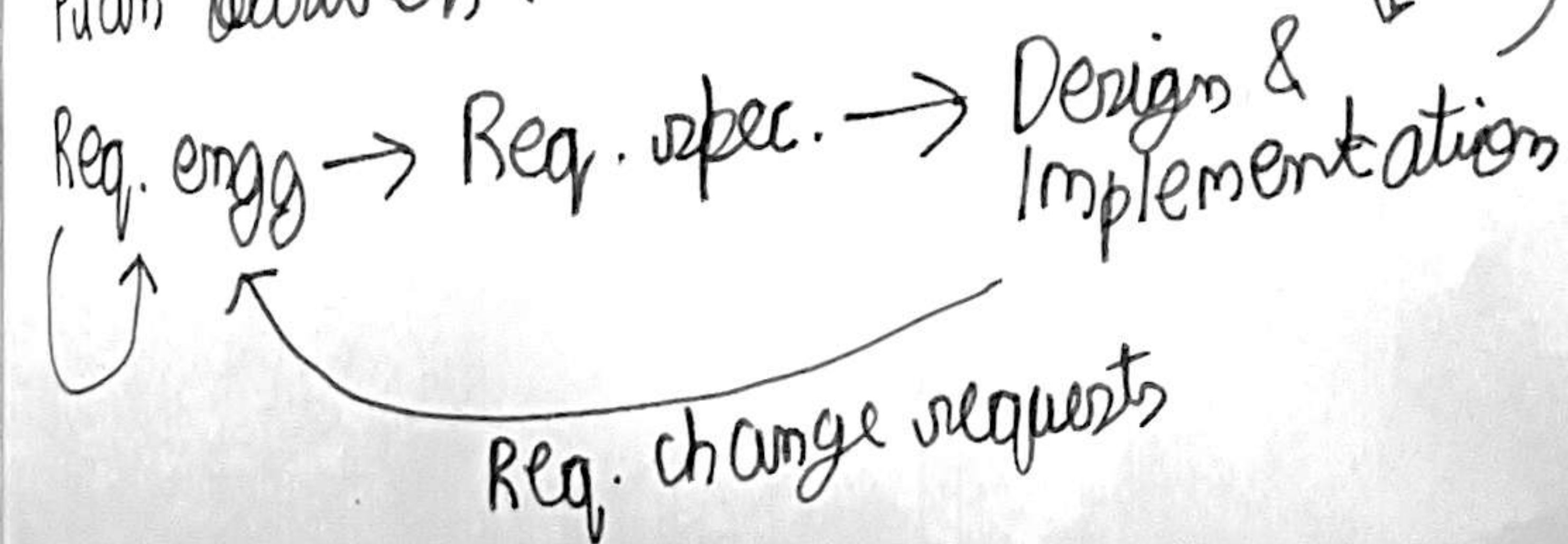
- Rapid development & delivery is now often the most important requirement for software systems
- Plan driven development is essential for some types of system but does not meet these business needs
- Agile development methods emerged in the late 1990's whose aim was to reduce the delivery time radically

## #Agile software development & characteristics -

- Program specification, design & implementation are interleaved
- System is developed as a series of versions or increments
- Frequent delivery of new versions for evaluation
- Extensive tool support used to support development
- Minimal documentation

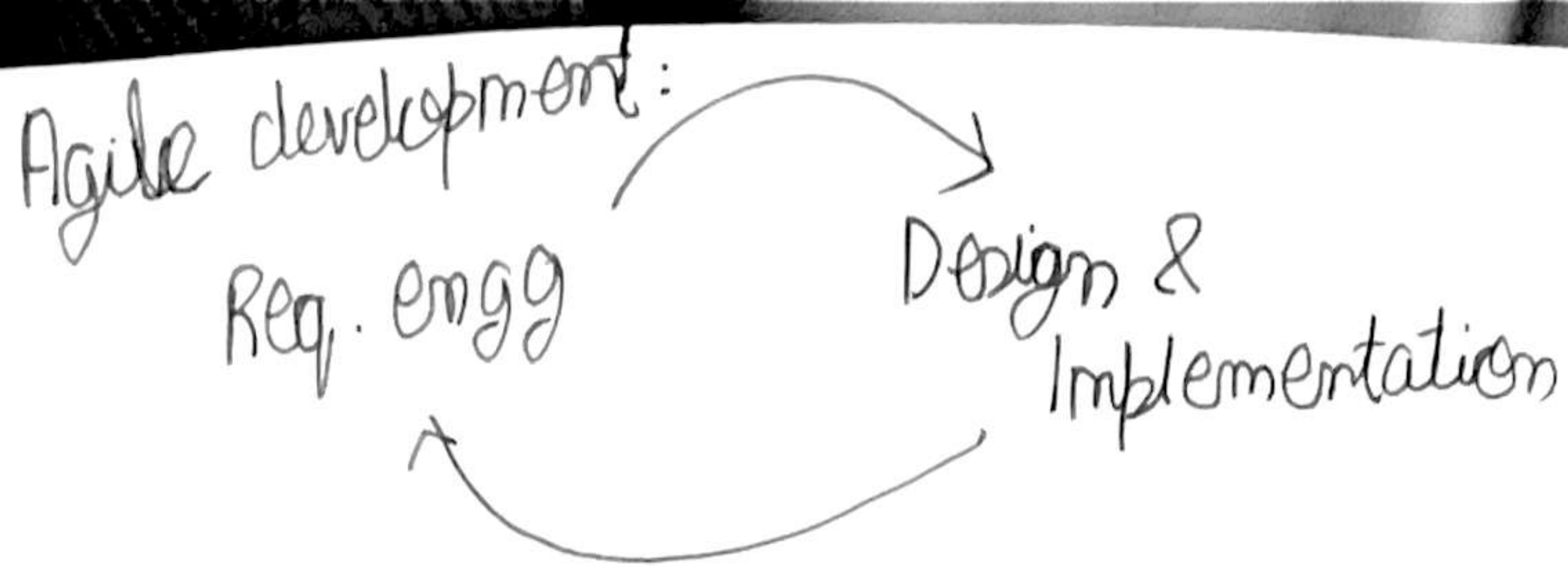
## #Plan driven VS Agile development -

Plan driven:



~~Design & Implementation~~





## # Agile methods:

- These methods:
  - (i) Focus on the code rather than the design
  - (ii) Are based on an iterative approach to software development
  - (iii) Are intended to deliver working products
- ~~Another~~ Aim is to reduce overheads in the software process (eg. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework

## Principles:

- (i) Customer ~~involved~~ involvement
- (ii) Incremental delivery
- (iii) People & process (No prescriptive processes)
- (iv) Embrace change
- (v) Maintain simplicity